



Module 05 - LLM Behavior Analysis & Evaluation Techniques

Building the Foundation for AI Engineering Excellence

Human-in-the-Loop Evaluation & Prompt-Level Tracing

Making LLM pipelines observable, collecting structured human feedback, and tracing prompts end-to-end to debug and systematically improve quality.



Today's Focus

Building Production-Grade LLM Observability

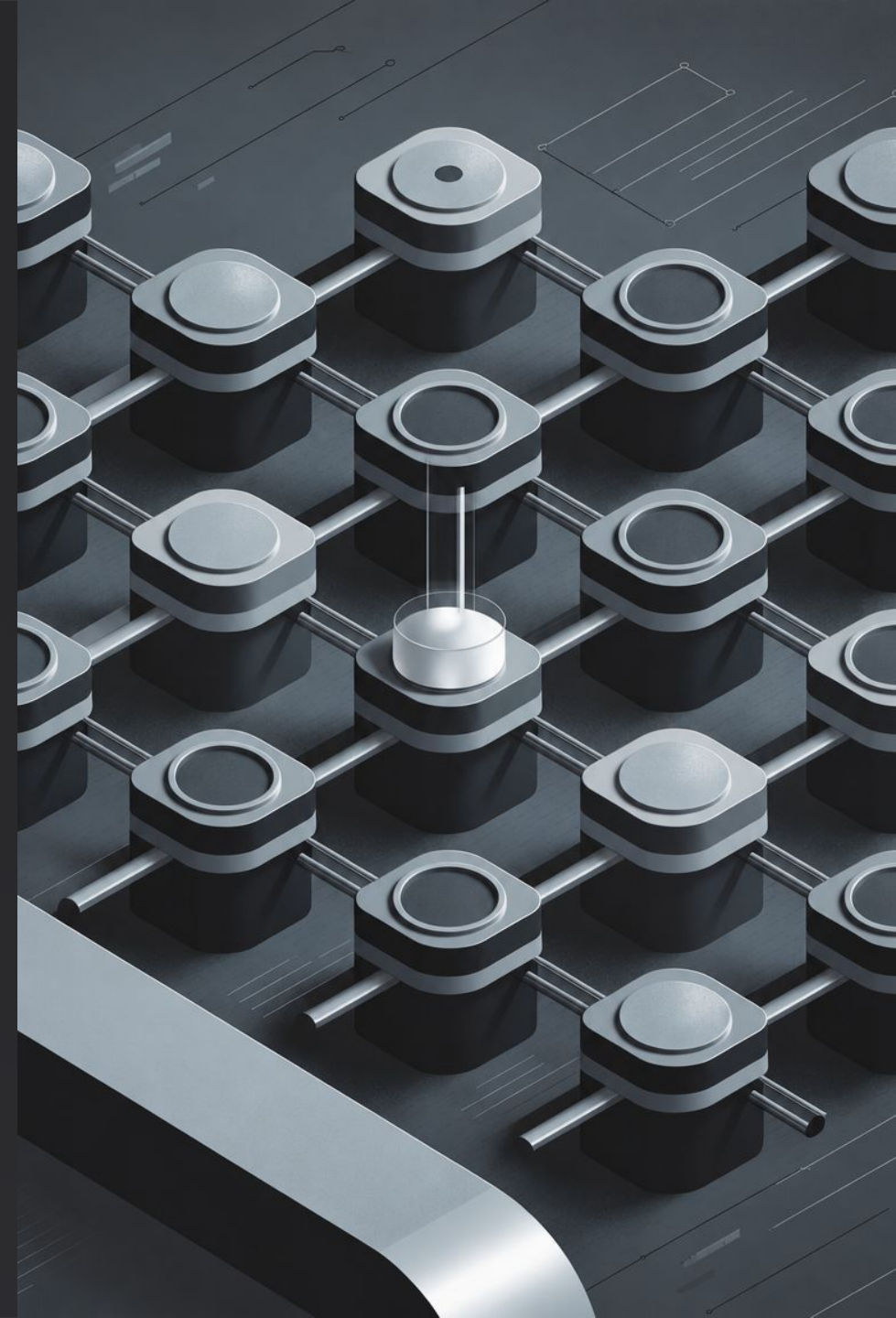
Modern LLM systems require more than just API calls and response parsing. To build reliable, production-ready applications, we need comprehensive observability: structured human feedback loops, detailed prompt-level tracing, and systematic evaluation frameworks. Today we'll explore the essential techniques for making your LLM pipelines transparent, debuggable, and continuously improvable.

These capabilities transform LLM development from experimental prototyping into engineering discipline—enabling you to measure quality, track regressions, investigate failures, and confidently deploy changes to production systems.

The Observability Challenge in LLM Systems

Unlike traditional software where bugs produce stack traces and deterministic outputs, LLM systems fail quietly and unpredictably. A prompt that worked yesterday might hallucinate today. A chain that succeeds on test data might produce biased outputs on real user queries. Without proper instrumentation, debugging becomes guesswork.

Production LLM systems require three pillars of observability: human evaluation to capture quality that metrics miss, detailed tracing to understand what actually happened during execution, and systematic datasets to measure improvements over time. Together, these enable the feedback loops necessary for continuous improvement.



What Is Human-in-the-Loop Evaluation?

The Core Concept

HITL evaluation systematically collects human judgments on LLM outputs using structured rubrics, rankings, or scoring systems.

Unlike automated metrics, humans can assess nuanced qualities like helpfulness, coherence, factual accuracy, and alignment with intent.

This isn't ad-hoc testing—it's a disciplined process with clear criteria, multiple annotators, and statistical analysis of agreement rates.

When to Use HITL

- Comparing prompt variants or model versions
- Evaluating subjective qualities (tone, clarity, helpfulness)
- Detecting subtle errors automated metrics miss
- Building gold-standard datasets for future automation
- Measuring improvement on ambiguous or creative tasks

Three Primary Feedback Collection Methods

Pairwise Ranking

Annotators compare two outputs and choose which is better. Simple, intuitive, and reduces cognitive load. Produces win rates and statistical significance through repeated comparisons.

Best for: A/B testing prompt changes, model selection, relative quality assessment

Likert Scoring

Rate outputs on scales (1-5 or 1-7) across multiple dimensions: accuracy, relevance, clarity, safety. Provides granular feedback and enables tracking improvement over time.

Best for: Multi-dimensional quality assessment, tracking score distributions, identifying specific weaknesses

Binary Accept/Reject

Simple thumbs up/down or pass/fail decisions. Fast to collect, easy to aggregate into acceptance rates. Can include optional reason tagging for failures.

Best for: Quality gates, production monitoring, quick feedback loops, regression testing

Designing Effective Evaluation Rubrics

A rubric transforms subjective judgment into measurable criteria. Poor rubrics lead to inconsistent annotations and unreliable results. Effective rubrics are specific, measurable, and aligned with your actual quality goals.

Be Specific and Operational

Instead of "good quality," define: "Response directly answers the question without hallucinating facts, uses appropriate technical depth, and provides actionable guidance." Every criterion should be checkable.

Include Examples and Edge Cases

Show annotators what a 5/5 looks like versus a 3/5. Document how to handle ambiguous cases. Calibrate the team on 20-30 examples before full annotation begins.

Track Multiple Dimensions

Separate scores for accuracy, completeness, safety, and tone reveal which aspects need improvement. A single aggregate score obscures the signal.



Inter-Annotator Agreement: Measuring Consistency

When multiple people evaluate the same outputs, how often do they agree? Inter-annotator agreement (IAA) quantifies this consistency and indicates rubric quality. Low agreement means unclear criteria or ambiguous tasks.

Cohen's Kappa measures agreement for two annotators on categorical judgments (accept/reject, class labels). Values above 0.7 indicate good agreement.

Fleiss' Kappa extends this to three or more annotators. Essential for larger annotation teams.

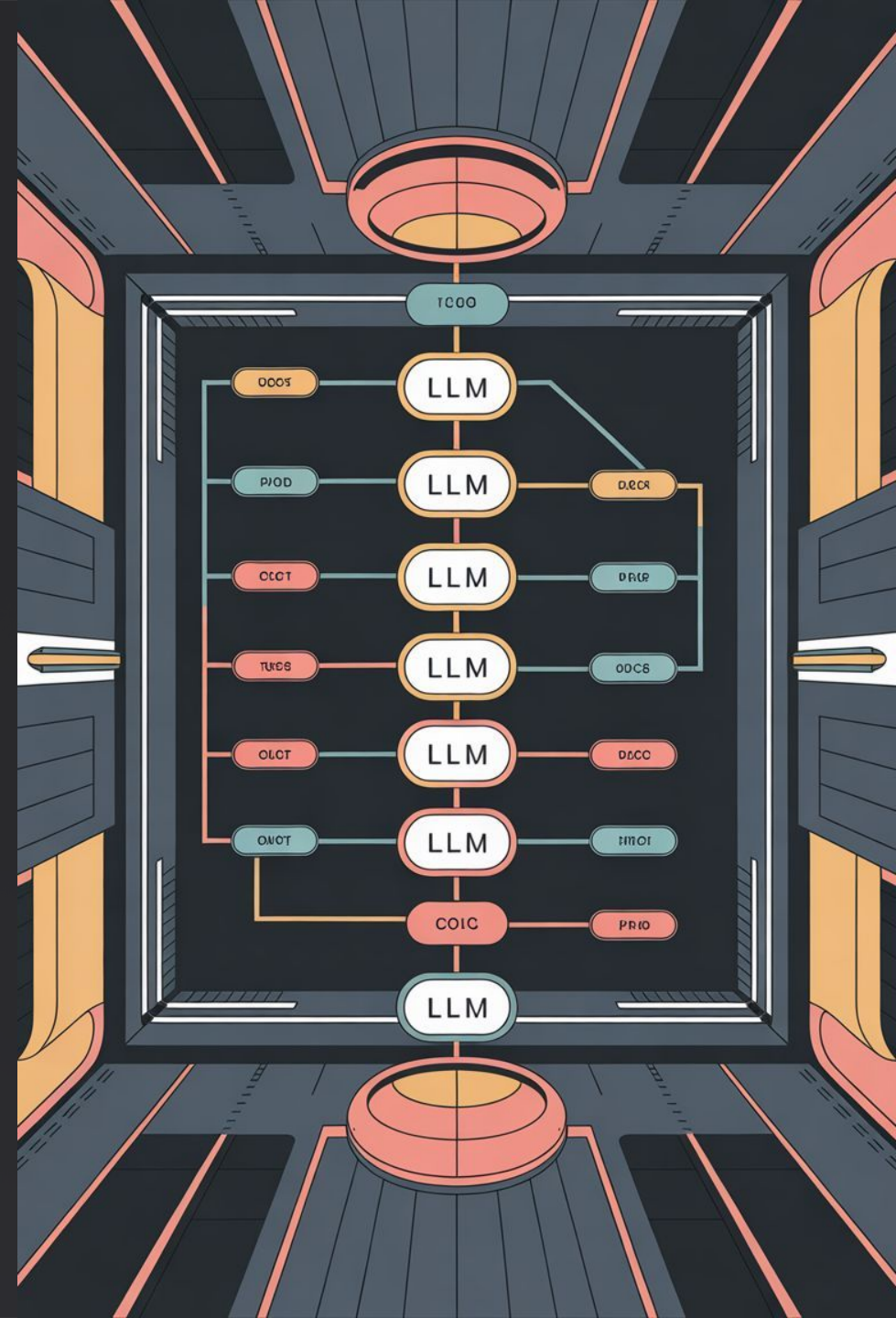
Pearson/Spearman correlation measures agreement on continuous scores (Likert scales). Check both correlation and score variance.

📌 **Rule of thumb:** If agreement is below 0.6, pause and refine your rubric. Interview annotators about ambiguous cases, add clarifying examples, and recalibrate before continuing.

What Is Prompt-Level Tracing?

Prompt-level tracing records the complete execution history of every LLM call in your pipeline: the exact prompt text with all variables filled in, model parameters, timestamps, latency, token counts, intermediate steps, tool calls, retries, and final outputs. Think of it as distributed tracing for LLM chains.

Without tracing, debugging is impossible. You see a bad output but don't know which prompt template was used, what variables were filled in, whether the model called a tool incorrectly, or if a retry changed the response. Tracing makes the entire pipeline transparent and reproducible.



Key Components of Comprehensive Tracing

01

Prompt Template & Variables

Capture the template string and all variable values. Save both the template itself and the final rendered prompt sent to the model.

02

Model Configuration

Record model name, temperature, max_tokens, top_p, and any other parameters. Configuration changes often cause unexpected behavior.

03

Intermediate Steps

For chains and agents, log each sub-call: retrieval results, tool invocations, parsing attempts, and decision points.

04

Outputs & Metadata

Store raw model output, parsed structured results, token counts, latency, cost estimates, and any error messages or warnings.

05

Context & Versioning

Tag each trace with prompt version ID, dataset name, commit SHA, user ID (if applicable), and session metadata for filtering and analysis.

LangSmith: Production Tracing for LLM Applications

LangSmith is a platform specifically designed for LLM observability. It automatically instruments LangChain applications but also works with raw OpenAI calls or custom frameworks via its Python SDK.

Key features include hierarchical trace views showing parent-child relationships between chain steps, automatic latency and cost tracking, dataset management, and comparison tools for A/B testing prompt variants.

```
import openai
from langsmith import trace
from langsmith.wrappers import wrap_openai

# Set these before any call.
# (LANGCHAIN_TRACING_V2 / LANGCHAIN_API_KEY still work as aliases.)
# LANGSMITH_TRACING=true
# LANGSMITH_API_KEY=ls_...

# Model in a config variable. gpt-4 is deprecated;
# confirm the current ID at build time.
MODEL = "gpt-5-mini"

# Wrap the OpenAI client so every call is auto-traced
client = wrap_openai(openai.Client())

# Group the call under one named, tagged parent span
with trace(
    name="summarize_paper",
    project_name="research-assistant",
    tags=["v2.3", "dataset:arxiv-cs"],
) as rt:
    response = client.chat.completions.create(
        model=MODEL,
        messages=[{"role": "user", "content": "..."}],
    )
```

LangSmith: Production Tracing for LLM Applications

Decorate form:

```
import openai
from langsmith import traceable
from langsmith.wrappers import wrap_openai

MODEL = "gpt-5-mini" # confirm current ID at build time
client = wrap_openai(openai.Client())

@traceable(name="summarize_paper", tags=["v2.3", "dataset:arxiv-cs"])
def summarize_paper(text: str) -> str:
    resp = client.chat.completions.create(
        model=MODEL,
        messages=[{"role": "user", "content": text}],
    )
    return resp.choices[0].message.content
```



Building Lineage Views for Debugging

A lineage view shows how data flows through your LLM pipeline from input to output. For a RAG system, this might be: user query → retrieval → reranking → context assembly → generation → post-processing. Each step is a node with inputs, outputs, and metadata.

When investigating a failure, lineage views answer critical questions: Did retrieval return irrelevant documents? Was the context too long and got truncated? Did a parsing step fail? Did the model refuse due to content policy? You can click into any node to see exact inputs and outputs, eliminating guesswork from debugging.

Tagging Strategies for Effective Trace Organization



Version Tags

Tag every trace with prompt version (e.g., "v2.4.1") and code commit SHA. When you discover a regression, you can instantly filter to the version that introduced it.



Dataset Tags

Identify which eval dataset or user cohort each trace belongs to. Essential for comparing performance across test sets and production segments.



User Context

Tag with `user_id`, `session_id`, or anonymized identifiers to investigate user-specific issues and track experience over sessions.



Outcome Tags

Mark traces as success/failure, thumbs-up/down, or with error categories. Enables filtering to problematic cases for root cause analysis.

Exporting Traces for Postmortem Analysis

When something goes wrong in production—a sudden spike in hallucinations, a drop in user satisfaction, or an outage—you need to export relevant traces for detailed investigation. Most tracing platforms support exporting via API or UI based on filters (time range, tags, error conditions).

What to Export

- All traces from the incident time window
- Traces with specific error tags or low scores
- Representative samples from before/after a deployment
- Complete traces for failed cases to reproduce locally

Analysis Techniques

- Compare prompt distributions before/after the incident
- Identify patterns in failing inputs or model responses
- Reconstruct exact failure conditions for debugging
- Build regression tests from failure cases

Evaluation Datasets: Gold Sets vs Living Sets

Gold Standard Sets

Carefully curated, stable datasets with expert annotations. Used for benchmarking and comparing across long time periods. Should represent diverse, challenging cases.

Size: 100-500 examples. Quality over quantity.

Update frequency: Rarely—only when task definition changes.

Living Evaluation Sets

Continuously updated with recent production examples, edge cases, and failure modes. Reflects actual usage patterns and evolves with your system.

Size: 1000+ examples, refreshed regularly.

Update frequency: Weekly or after significant product changes.



Stratified Sampling for Representative Datasets

Random sampling from production data often misses rare but important cases. Stratified sampling ensures your eval set represents key segments: different query types, user intents, difficulty levels, and edge cases.



Define Strata

Identify categories that matter: query complexity, topic, user segment, expected difficulty. For a code assistant: beginner questions, API lookups, debugging, architecture design.



Set Proportions

Decide how many examples per stratum. Often overweight rare but critical cases (safety issues, failure modes) relative to their production frequency.



Sample & Validate

Randomly sample within each stratum. Validate coverage and remove duplicates or low-quality examples before annotation.

Running A/B Experiments with Prompt Variants

Once you have tracing and datasets in place, you can run rigorous A/B experiments. Deploy prompt variant A to 50% of traces and variant B to the other 50%, both tagged appropriately. After collecting sufficient data, compare outcomes using statistical tests.

Key metrics to track include win rate (if doing pairwise comparisons), average Likert scores across dimensions, acceptance rate, latency, token usage, and cost per query. Don't just look at averages—examine distributions and outliers.

Use confidence intervals and hypothesis tests (t-test, Mann-Whitney U, or bootstrap) to determine if differences are statistically significant. A 2% improvement might not be meaningful if your sample size is small.

📄 **Sample size rule:** For detecting a 5% difference in acceptance rate with 80% power and $\alpha=0.05$, you need ~1000 samples per variant. Use power analysis calculators before running experiments.

Regression Gates: Preventing Quality Backslides

A regression gate is an automated check that runs your eval dataset through a new prompt version and verifies that quality metrics haven't degraded below thresholds. This prevents accidentally shipping prompts that perform worse than the current production version.

1

Commit

Developer updates prompt template and commits to version control

2

CI Triggers Eval

Continuous integration runs the new prompt on gold standard dataset

3

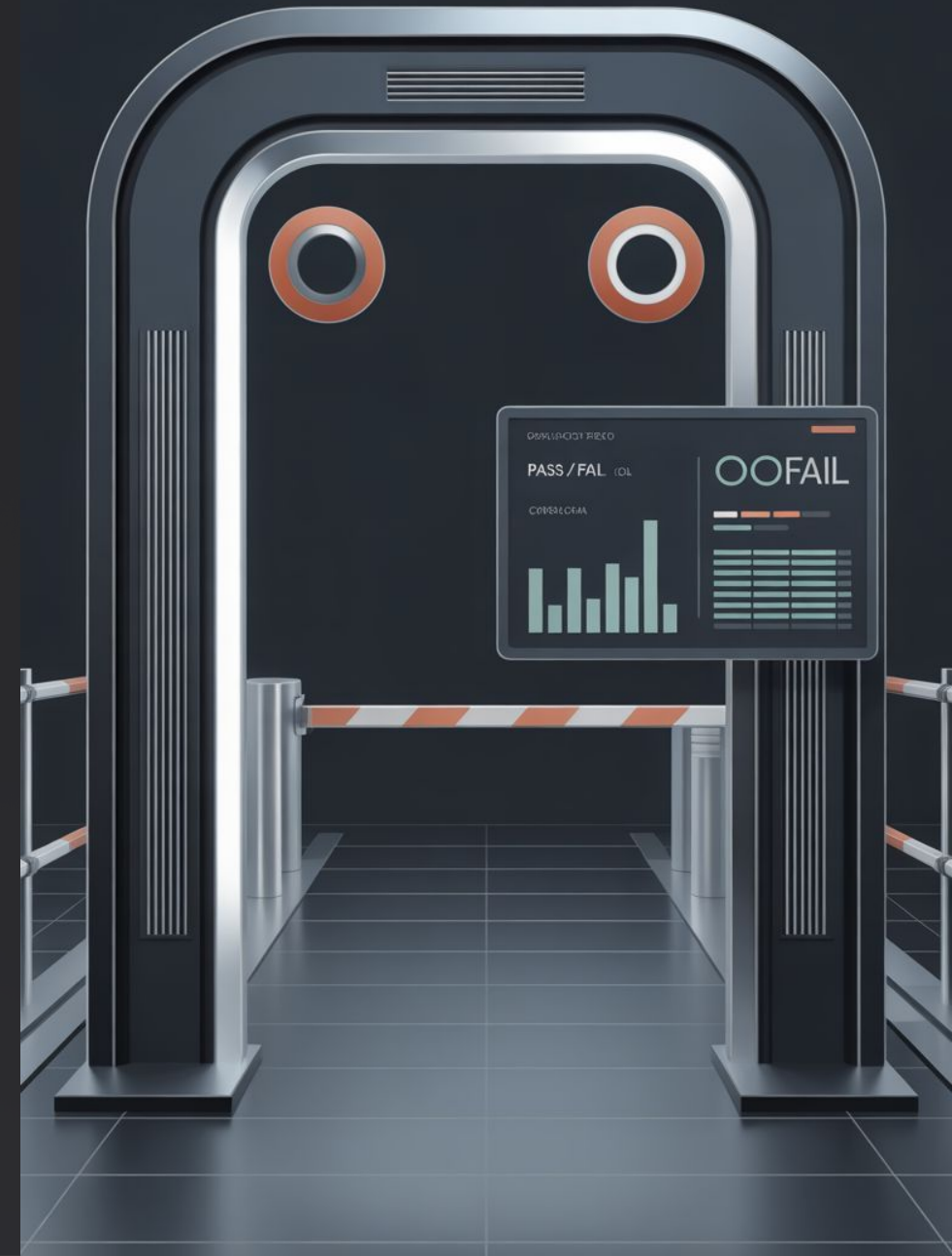
Compare Metrics

Automated comparison against baseline: accuracy, relevance, safety scores

4

Gate Decision

If metrics exceed thresholds, test passes. Otherwise, block deployment.



Setting Appropriate Regression Thresholds

Absolute Thresholds

Set minimum acceptable values: accuracy ≥ 0.85 , safety score $\geq 4.5/5$, hallucination rate $\leq 5\%$. Simple and interpretable, but doesn't account for baseline variance.

Best for: Critical safety or accuracy requirements that must never be violated.

Relative Thresholds

Compare to production baseline: new version must be within -2% of current accuracy, no worse than +50ms latency. Adapts as overall quality improves.

Best for: Iterative improvements where you want to prevent backsliding while allowing experimentation.

In practice, use both: absolute thresholds for must-have qualities (safety, basic coherence) and relative thresholds for competitive metrics (helpfulness, style). Build in acceptable ranges—not every change needs to improve every metric.

Win/Loss Analysis: Learning from Experiments

When comparing two prompt variants, aggregate metrics only tell part of the story. Win/loss analysis examines individual examples where variant A beat variant B (wins) and where B beat A (losses). This reveals what changed and why.

1

Identify Wins & Losses

For each example in your eval set, determine which variant produced the better output based on your rubric.

2

Cluster Patterns

Group wins and losses by input characteristics: query type, complexity, topic. What kinds of inputs does the new prompt handle better?

3

Investigate Examples

Manually review a sample of wins and losses. What did the prompt change improve? What regressions occurred?

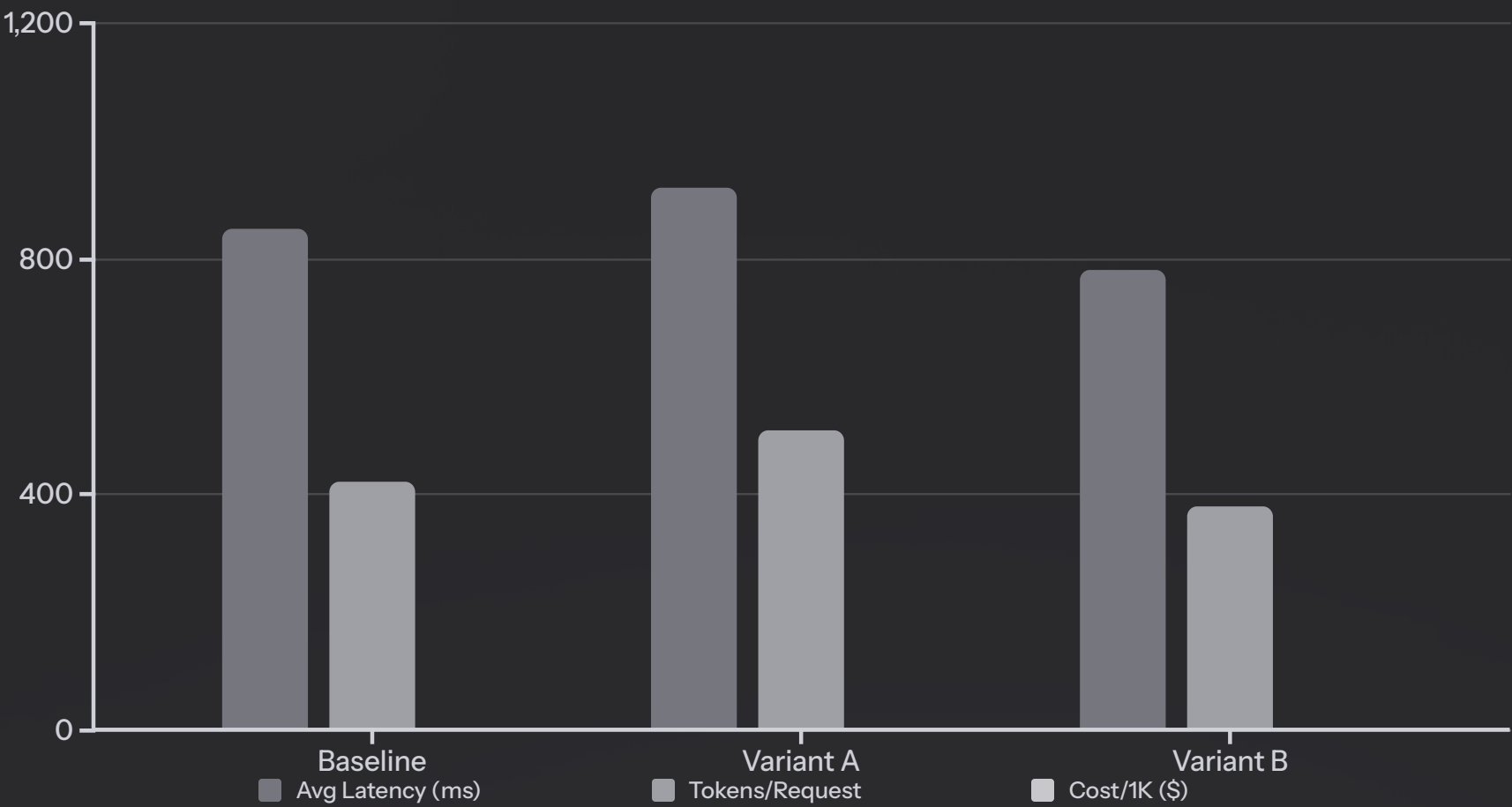
4

Iterate

Use insights to refine the prompt further or decide if the trade-offs are acceptable for deployment.

Cost and Latency Tracking in Evaluation

Quality isn't the only dimension that matters—cost and latency are first-class concerns in production systems. A prompt that improves accuracy by 3% but doubles latency or triples token usage may not be deployable. Track these metrics alongside quality.



Variant B improves both latency and cost, making it attractive even if quality gains are modest. Multi-objective optimization requires weighing trade-offs explicitly.



Confidence Intervals and Statistical Significance

With small eval sets, metrics are noisy. An observed 3% improvement might be due to chance. Confidence intervals quantify uncertainty, and statistical tests determine if differences are likely real or just sampling variance.

Bootstrap Confidence Intervals

Resample your eval set with replacement 1000+ times, recompute metrics, and take the 2.5th and 97.5th percentiles. This gives a 95% CI without distributional assumptions.

If the CIs for two variants don't overlap, the difference is likely significant.

Hypothesis Testing

For binary outcomes (accept/reject), use a two-proportion z-test. For continuous scores, use a t-test or Mann-Whitney U test if distributions are non-normal.

Set $\alpha=0.05$ for significance. Report p-values alongside effect sizes.

Real-World Example: Document Q&A System

Consider a retrieval-augmented Q&A system over technical documentation. You want to improve answer quality by refining the prompt that synthesizes retrieved chunks into a response. Here's how you'd apply HITL evaluation and tracing:

01	02	03
Instrument with Tracing	Create Eval Dataset	Run Baseline & Variant
Add LangSmith tracing to capture retrieval queries, retrieved chunks, reranking scores, final prompt, and model output. Tag with prompt version and eval dataset ID.	Curate 200 real user questions stratified by complexity (factual lookup, conceptual explanation, multi-hop reasoning). Include ground truth or expert-written ideal answers.	Generate answers with current production prompt (baseline) and new candidate prompt. Trace both runs.
04	05	
HITL Annotation	Analyze & Decide	
Have 3 annotators score each answer on accuracy (1-5), completeness (1-5), and clarity (1-5) using a detailed rubric. Compute inter-annotator agreement.	Compare mean scores, perform t-tests, examine win/loss examples, check latency/cost. If variant is significantly better and passes regression gates, deploy.	

Building a Simple HITL Annotation Interface

You don't need a complex platform to start collecting human feedback. A Jupyter notebook with ipywidgets or a minimal Streamlit app can work for small teams. The key elements: display the input, show outputs side-by-side or sequentially, provide scoring controls (radio buttons, sliders), and save annotations to a structured format (JSON, CSV, database).

```
import streamlit as st
import json

# Load eval examples
examples = json.load(open("eval_set.json"))

# Simple pairwise ranking UI
for i, ex in enumerate(examples):
    st.write(f"Question: {ex['question']}")
    col1, col2 = st.columns(2)
    with col1:
        st.write("Output A:")
        st.write(ex['output_a'])
    with col2:
        st.write("Output B:")
        st.write(ex['output_b'])

    choice = st.radio(f"Which is better? (Ex {i})",
        ["A", "B", "Tie"], key=f"choice_{i}")
    st.divider()
```

Aggregating Annotation Data and Computing Metrics

Once annotations are collected, aggregate them to produce metrics. For pairwise rankings, count how often each variant won and compute a win rate with confidence interval. For Likert scores, compute mean and median across annotators and examples, and check distributions for each dimension.

Win Rate Calculation

If variant A wins 65 out of 100 comparisons, its win rate is 65%. Use a binomial test to check if this is significantly different from 50% (random chance).

Bootstrap the win rate by resampling annotations to get a 95% CI, e.g., [56%, 73%].

Score Aggregation

Average Likert scores across annotators first, then across examples. Report mean, median, and standard deviation. Check for ceiling/floor effects (scores bunched at extremes).

Use Krippendorff's alpha to measure agreement on ordinal scales.



Investigating Disagreements Among Annotators

When annotators disagree, don't just average their scores—investigate why. Disagreements often reveal ambiguous rubric criteria, genuinely subjective cases, or annotation errors. Review high-disagreement examples as a team, discuss reasoning, and refine the rubric.

- **Calculate per-example variance**

Identify examples where annotator scores differ by 2+ points. These are disagreement hotspots.

- **Review and discuss**

Bring annotators together to review disagreement cases. Often one person missed a detail or interpreted the rubric differently.

- **Refine rubric**

Add clarifying examples, edge case handling, or split ambiguous criteria into multiple dimensions.

- **Re-annotate if needed**

After rubric updates, re-annotate a subset to confirm improved agreement before continuing full annotation.

Writing Prompt Release Notes

When you deploy a new prompt version, document what changed and why in a release note. This creates institutional memory, helps future debugging, and communicates progress to stakeholders. A good release note includes version number, what changed (prompt text, model, parameters), eval results (win rate, score improvements, latency/cost impact), known limitations or regressions, and links to traces and experiments.

Example Release Note

****Prompt v3.2.1 - Document Q&A****

Released: 2024-01-15

****Changes:****

- Added explicit instruction to cite sources
- Reduced temperature: 0.7 → 0.5

****Eval Results (n=200):****

- Accuracy: 4.2 → 4.5 (+0.3, p<0.01)
- Completeness: 4.1 → 4.3 (p=0.08)
- Latency: 820ms → 780ms

****Regressions:****

- Slightly more conservative on ambiguous queries

****Deployment:**** Staged rollout,
10% → 50% → 100% over 48hrs

This format communicates key information concisely. Stakeholders see improvements, engineers can debug future issues by referencing version history, and the team builds a shared understanding of quality evolution over time.

Advanced Techniques: Active Learning for Annotation

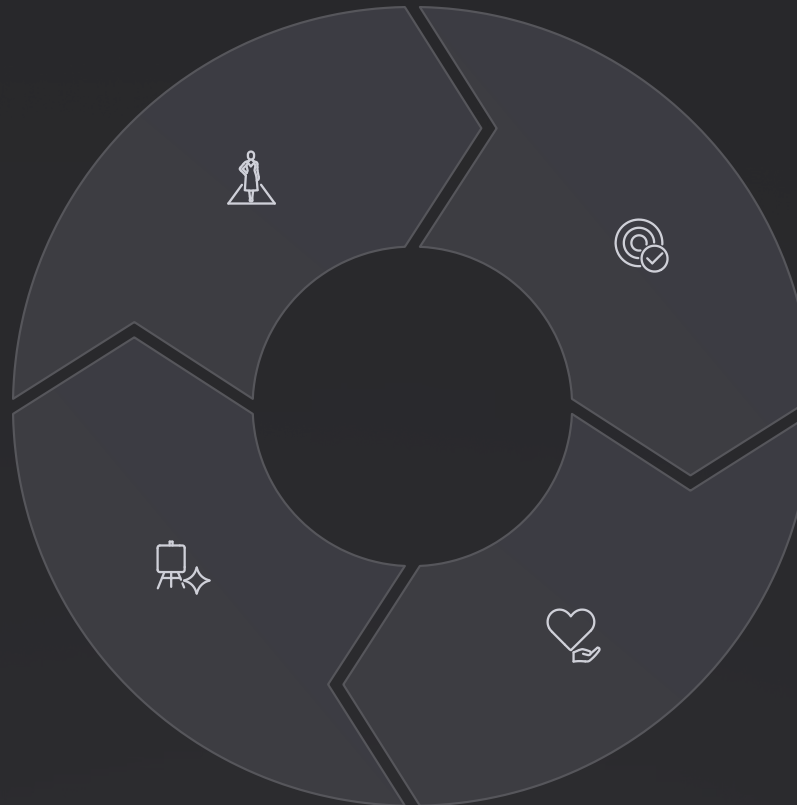
Annotating thousands of examples is expensive. Active learning helps prioritize which examples to annotate by identifying the most informative ones: cases where the model is uncertain, recent failures, edge cases, or examples where two variants disagree. This maximizes learning per annotation dollar.

Model Inference

Run your LLM pipeline on a large unlabeled dataset, capturing confidence scores or uncertainty estimates.

Update Eval Set

Add annotated examples to your evaluation dataset and repeat the cycle, continuously improving coverage.

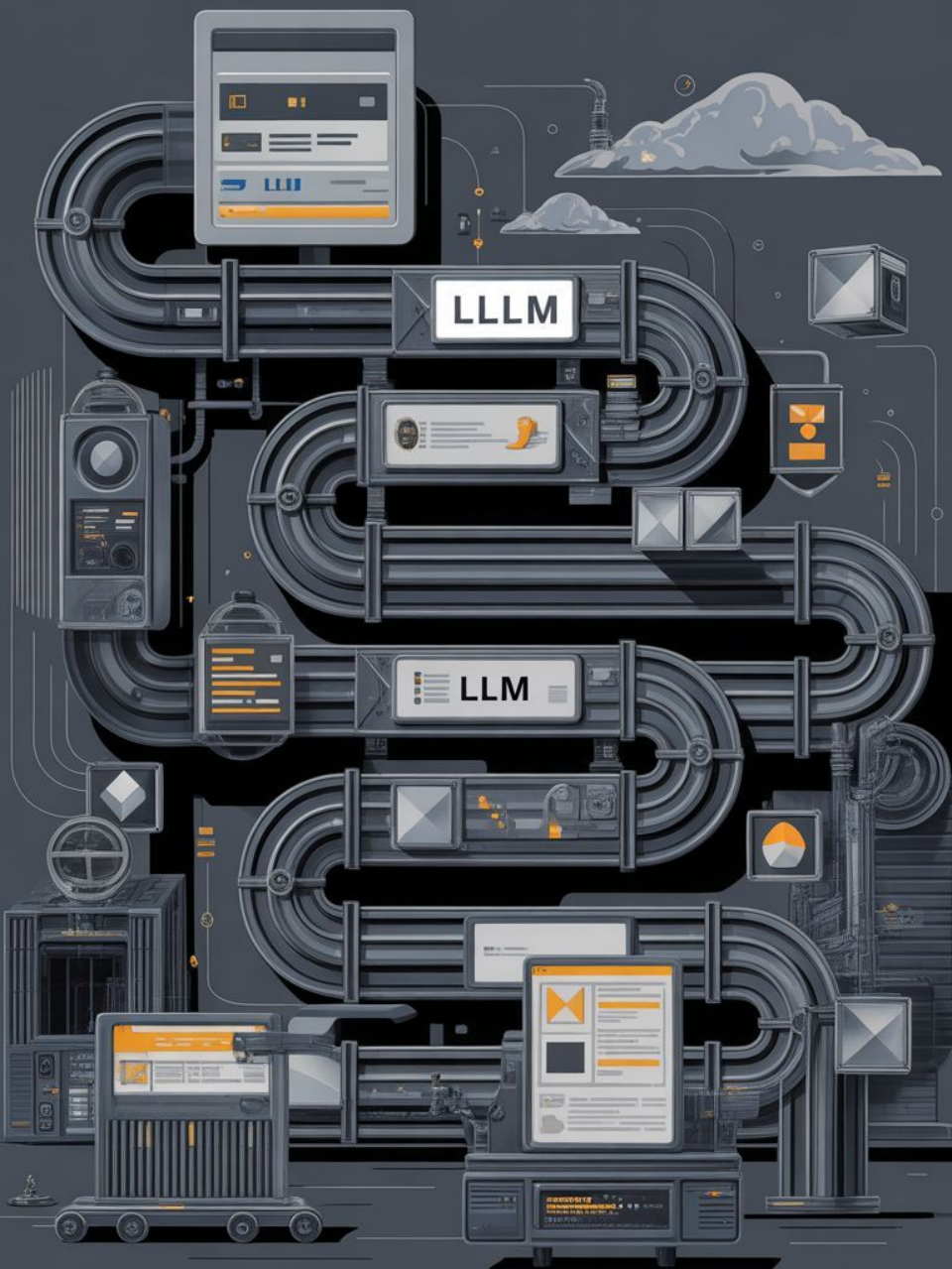


Selection Strategy

Rank examples by informativeness: low-confidence predictions, high disagreement between variants, or diversity sampling.

Annotate Batch

Human annotators label the selected batch using your rubric, focusing effort where it matters most.



Integration with CI/CD Pipelines

To make evaluation and tracing part of your development workflow, integrate them into CI/CD. On every pull request, automatically run the new prompt on your gold standard dataset, compare metrics to baseline, and post results as a comment. Block merges if regression gates fail.

GitHub Actions Example

Trigger an evaluation job on PR creation. The job checks out code, installs dependencies, runs the eval script against LangSmith-traced prompts, computes metrics, and posts results to the PR. If metrics fail thresholds, the check fails and the PR can't merge.

Benefits

- Catch regressions before they reach production
- Make quality visible to reviewers
- Enforce consistent evaluation standards
- Build confidence in deployment decisions

Common Pitfalls and How to Avoid Them

Overfitting to Eval Set

If you optimize prompts by repeatedly testing on the same eval set, you'll overfit. Maintain a held-out test set that's never used for development, only for final validation before deployment.

Ignoring Distribution Shift

Eval sets curated months ago may not reflect current user queries. Regularly refresh your datasets with recent production examples to stay aligned with actual usage.

Insufficient Sample Sizes

Small eval sets ($n < 50$) yield noisy metrics and spurious differences. Invest in larger datasets and use statistical tests to validate significance before making decisions.

Poor Rubric Design

Vague criteria lead to inconsistent annotations. Spend time upfront defining clear, measurable rubrics with examples. Check inter-annotator agreement early and iterate.

Key Takeaways: Building Observable LLM Systems



HITL Evaluation is Essential

Automated metrics miss nuanced quality issues. Structured human feedback via pairwise ranking, Likert scoring, or binary judgments provides the signal needed for continuous improvement.



Trace Everything

Comprehensive prompt-level tracing makes LLM systems debuggable and reproducible. Capture inputs, outputs, intermediate steps, metadata, and version tags for every execution.



Systematic Experimentation

Use stratified datasets, A/B testing, regression gates, and statistical analysis to rigorously evaluate changes. Avoid deploying based on anecdotes or cherry-picked examples.



Integrate into Workflow

Make evaluation and tracing part of your CI/CD pipeline. Automated checks prevent regressions and build confidence in deployment decisions.

With these practices, you transform LLM development from art into engineering—enabling you to measure quality, understand failures, and systematically improve production systems.